



برنامه نویسی به زبان سی

پردازش فایل

دکتر مهران صفایانی

safayani@iut.ac.ir

safayani.iut.ac.ir



<https://www.aparat.com/mehran.safayani>



https://github.com/safayani/Programming_Basics_course



File Processing in C

- **Files** enable **long-term data retention** after a program ends.
- Data stored on secondary storage (SSDs, flash drives, hard drives).
- Each file is a **sequential stream of bytes**.



FILE Structure and File Pointers

- **FILE Structure (<stdio.h>)**
 - fopen() returns a **pointer to a FILE structure**.
 - Contains system info for processing the file (e.g., file descriptor, file control block).
- **File Pointers**
 - stdin, stdout– pointers to standard streams.
 - Each open file has its own FILE pointer.

Common File-Processing Functions

- **Character I/O**

- `fgetc(filePtr)` – Reads one character (like `getchar()`).
- `fputc(char, filePtr)` – Writes one character (like `putc()`).

- **String and Formatted I/O**

- `fgets()` / `fputs()` – Read/write a line of text.
- `fscanf()` / `fprintf()` – Formatted input/output to files.
- `fread()` / `fwrite()` – For binary data (discussed later).

Creating a Sequential-Access File

- **Overview**

- C imposes **no structure** on files; you define the record format.
- Example: Accounts receivable system (account number, name, balance).

- **Record key:** Account number.

- **Steps**

- Define a FILE pointer.
- Open the file with fopen().
- Write data using fprintf().
- Close the file with fclose().

Opening and Closing Files

- `fopen()` – **Opening a File**

```
FILE *cfPtr = fopen("clients.txt", "w");
```

- **Arguments:** Filename (with optional path) and file-open mode.
- **Mode "w":** Opens for writing. Creates file if nonexistent; **overwrites** if existing.
- Returns NULL if file cannot be opened.
- `fclose()` – **Closing a File**

```
fclose(cfPtr);
```

- Frees system resources.
- Always close files when no longer needed.

Checking for End-of-File (EOF)

- **feof() Function**
- Checks if end-of-file indicator is set for a stream.
- Example:

```
while (!feof(stdin)) { ... }
```

- **EOF key combinations:**
 - **Windows:** Ctrl + Z then Enter
 - **macOS/Linux:** Ctrl + D

Writing to a File with fprintf()

- fprintf() – **Formatted File Output**
- Similar to printf(), but includes a FILE pointer argument.
- Example:

```
fprintf(cfPtr, "%d %s %.2f\n", account, name, balance);
```

- Can also write to standard output using stdout.

Text Modes

Mode	Name	Read	Write	Append	File Exists	File Doesn't Exist	Position After Open	Exclusive
"r"	Read	 Yes	 No	 No	Opens file	Error (returns NULL)	Beginning	 No
"w"	Write	 No	 Yes	 No	Truncates to zero	Creates new	Beginning	 No
"a"	Append	 No	 Yes	 Yes	Preserves content	Creates new	End (writing) Beginning (reading with seek)	 No
"r+"	Read/Write	 Yes	 Yes	 No	Opens file	Error (returns NULL)	Beginning	 No
"w+"	Write/Read	 Yes	 Yes	 No	Truncates to zero	Creates new	Beginning	 No
"a+"	Read/Append	 Yes	 Yes	 Yes	Preserves content	Creates new	End (writing) Beginning (reading with seek)	 No
"wx"	Exclusive Write	 No	 Yes	 No	Error (returns NULL)	Creates new	Beginning	 Yes
"w+x"	Exclusive Write/Read	 Yes	 Yes	 No	Error (returns NULL)	Creates new	Beginning	 Yes

Binary Modes

Mode	Name	Read	Write	Append	File Exists	File Doesn't Exist	Position After Open	Exclusive
"rb"	Read Binary	✔ Yes	✘ No	✘ No	Opens file	Error (returns NULL)	Beginning	✘ No
"wb"	Write Binary	✘ No	✔ Yes	✘ No	Truncates to zero	Creates new	Beginning	✘ No
"ab"	Append Binary	✘ No	✔ Yes	✔ Yes	Preserves content	Creates new	End (writing) Beginning (reading with seek)	✘ No
"rb+" / "r+b"	Read/Write Binary	✔ Yes	✔ Yes	✘ No	Opens file	Error (returns NULL)	Beginning	✘ No
"wb+" / "w+b"	Write/Read Binary	✔ Yes	✔ Yes	✘ No	Truncates to zero	Creates new	Beginning	✘ No
"ab+" / "a+b"	Read/Append Binary	✔ Yes	✔ Yes	✔ Yes	Preserves content	Creates new	End (writing) Beginning (reading with seek)	✘ No
"wbx"	Exclusive Write Binary	✘ No	✔ Yes	✘ No	Error (returns NULL)	Creates new	Beginning	✔ Yes
"wb+x" / "w+b x"	Exclusive Write/Read Binary	✔ Yes	✔ Yes	✘ No	Error (returns NULL)	Creates new	Beginning	✔ Yes

Common File-Processing Errors

- **Typical Errors**

- Opening a **nonexistent file** for reading.
- **Insufficient permissions** to access the file.
- **Insufficient disk space** for writing.
- Opening in "w" mode **accidentally overwriting** an existing file.

- **Best Practices**

- **Always check** if fopen() returns NULL.
- **Close files** explicitly when done.
- Use **exclusive mode ("wx")** to prevent accidental overwrites.
- Test file operations in a controlled environment.

```
#include <stdio.h>
int main(void) {
    FILE *cfPtr = fopen("clients.txt", "w");
    if (cfPtr == NULL) { puts("File could not be opened"); return 1; }
    puts("Enter the account, name, and balance.");
    puts("Enter EOF (Ctrl+Z then Enter) to end input.");
    printf("? ");
    int account;
    char name[30];
    double balance;
    while (scanf("%d %29s %lf", &account, name, &balance) == 3) {
        fprintf(cfPtr, "%d %s %.2f\n", account, name, balance);
        printf("? "); }
    fclose(cfPtr);
    return 0;}
```

Reading Data from Sequential-Access Files

- *// Open file with minimum required permissions*

`FILE *file = fopen("data.txt", "r");` *// Read-only mode*

- **Read-only mode ("r"):** Prevents accidental modifications
- **Security best practice:** Only grant necessary access
- **Error prevention:** Cannot overwrite data accidentally
- **Reading vs Writing Mode**
- **Writing:** "w", "a", "r+" - Can modify
- **Reading:** "r" - View only, safe access

Reading with fscanf()

- *// Syntax: Similar to scanf()*
- `int fscanf(FILE *stream, const char *format, ...);`
- *// Example from Fig. 11.2*
- `FILE *cfPtr = fopen("clients.txt", "r");`
- `fscanf(cfPtr, "%d%s%lf", &account, name, &balance);`
- **Key Characteristics**
- **First argument:** FILE pointer (not stdin)
- **Format identical** to scanf()
- **Returns number** of successfully read items
- **Returns EOF** on end-of-file or error

```

1 // fig11_02.c
2 // Reading and printing a sequential file
3 #include <stdio.h>
4
5 int main(void) {
6     FILE *cfPtr = NULL; // cfPtr = clients.txt file pointer
7
8     // fopen opens file; exits program if file cannot be opened
9     if ((cfPtr = fopen("clients.txt", "r")) == NULL) {
10         puts("File could not be opened");
11     }
12     else { // read account, name and balance from file
13         int account = 0; // account number
14         char name[30] = ""; // account name
15         double balance = 0.0; // account balance
16
17         printf("%-10s%-13s%\n", "Account", "Name", "Balance");
18         fscanf(cfPtr, "%d%29s%lf", &account, name, &balance);
19
20         // while not end of file
21         while (!feof(cfPtr)) {
22             printf("%-10d%-13s%7.2f\n", account, name, balance);
23             fscanf(cfPtr, "%d%29s%lf", &account, name, &balance);
24         }
25
26         fclose(cfPtr); // fclose closes the file
27     }
28 }

```

Account	Name	Balance
100	Jones	24.98
200	Doe	345.67
300	White	0.00
400	Stone	-42.16
500	Rich	224.62

rewind() Function

- *// Reposition to beginning of file*

`rewind(cfPtr);`

- **Use Cases**

- Process file multiple times
- Start reading from beginning after EOF
- Reset for re-scanning operations


```
#include <stdio.h>
int main() {
    FILE *f = fopen("demo.txt", "r+");
    // Write some numbers
    for (int i = 1; i <= 5; i++) {
        fprintf(f, "%d ", i);  }
    rewind(f); // Back to start Read them back
    int num;
    printf("Numbers: ");
    while (fscanf(f, "%d", &num) == 1) {
        printf("%d ", num);
    }
    fclose(f);
    return 0;
}
```

```
Reading file first time:
Line 1
Line 2
Line 3

Reading file second time (after rewind):
Line 1
Line 2
Line 3
```

Random-Access Files

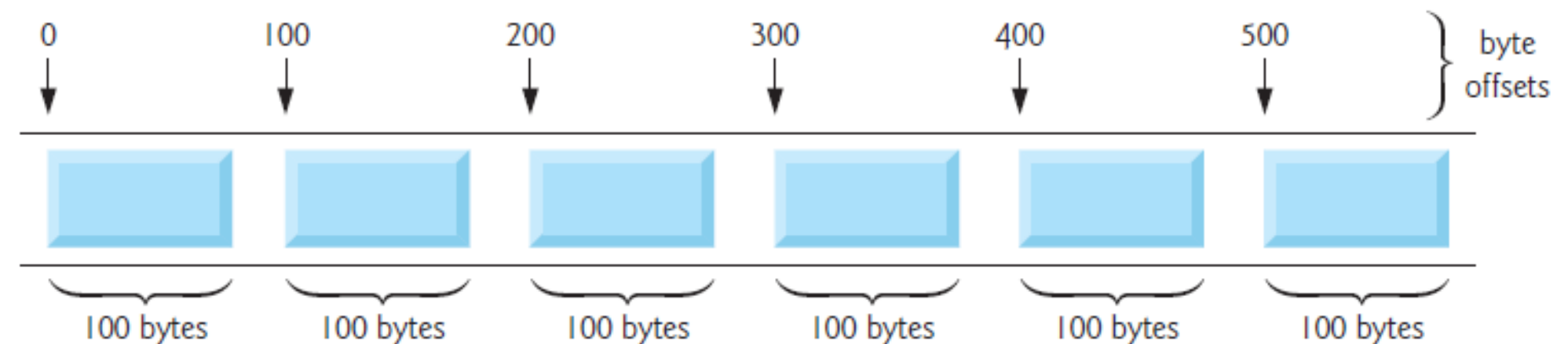
- Fixed-length records for direct access
- Transaction processing applications
- Binary file operations with fread/fwrite
- $\text{Record Location} = \text{Record Number} \times \text{Record Size}$

- **Example:**

- Record size = 100 bytes
- Record #5 location = $5 \times 100 = \text{byte } 500$
- Can jump directly to byte 500!

- **Benefits**

- **Immediate access** to any record
- **No searching** through other records
- **Safe updates** without affecting other data
- **Efficient** for large files



Binary File Operations

- **fwrite() Function**

// Writes binary data to file

```
size_t fwrite(const void *ptr, size_t size, size_t count, FILE *stream);
```

- **fread() Function**

// Reads binary data from file

```
size_t fread(void *ptr, size_t size, size_t count, FILE *stream);
```

- **Arguments**

- ptr - Pointer to data/memory location
- size - Size of each element (bytes)
- count - Number of elements
- stream - FILE pointer

Text vs Binary Output

- **Text Output (fprintf)**

```
int number = 1234567890;
```

```
fprintf(fPtr, "%d", number);
```

```
// Could write 10 characters = ~10 bytes
```

```
// Human-readable: "1234567890"
```

- **Binary Output (fwrite)**

```
int number = 1234567890;
```

```
fwrite(&number, sizeof(int), 1, fPtr);
```

```
// Always writes sizeof(int) bytes (typically 4)
```

```
// System-dependent binary format
```

Example Data Structure

- Create transaction system with:
 - **100 fixed-length records**
 - **Account number** (record key), **Last name**, **First name**, **Balance**
 - Operations: Update, Insert, Delete, List

- **Struct Definition**

```
struct clientData {  
    unsigned int accountNum; // 4 bytes  
    char lastName[15];      // 15 bytes  
    char firstName[10];     // 10 bytes  
    double balance;         // 8 bytes (typically)  
}; // Total: ~37 bytes fixed length
```

-


```

1 // fig11_04.c
2 // Creating a random-access file sequentially
3 #include <stdio.h>
4
5 // clientData structure definition
6 struct clientData {
7     int account;
8     char lastName[15];
9     char firstName[10];
10    double balance;
11 };
12
13 int main(void) {
14     FILE *cfPtr = NULL; // accounts.dat file pointer
15
16     // fopen opens the file; exits if file cannot be opened
17     if ((cfPtr = fopen("accounts.dat", "wb")) == NULL) {
18         puts("File could not be opened.");
19     }
20     else {
21         // create clientData with default information
22         struct clientData blankClient = {0, "", "", 0.0};
23
24         // output 100 blank records to file
25         for (int i = 1; i <= 100; ++i) {
26             fwrite(&blankClient, sizeof(struct clientData), 1, cfPtr);
27         }
28
29         fclose (cfPtr); // fclose closes the file
30     }
31 }

```

What is fseek()?

- Changes the file position indicator (File Pointer)
- Enables random access to files
- Move forward/backward within a file
- Jump to specific positions

```
int fseek(FILE *stream, long offset, int whence);
```

- stream: Pointer to FILE object
- offset: Number of bytes to offset
- whence: Starting position for the offset
- **Return Value**
 - **0** on success
 - **Non-zero** on failure (e.g., seeking before file start)

The whence Parameter

Value	Standard Constant	Description	Example
0	SEEK_SET	From beginning of file	fseek(fp, 100, SEEK_SET)
1	SEEK_CUR	From current position	fseek(fp, -10, SEEK_CUR)
2	SEEK_END	From end of file	fseek(fp, -50, SEEK_END)

Example:

```
fseek(cfPtr, offset, SEEK_SET); // Position pointer  
fwrite(&client, sizeof(struct clientData), 1, cfPtr); // Write
```


Positioning with fseek()

- **Position Calculation Formula**

// For account numbers 1-100:

*offset = (client.account - 1) * sizeof(struct clientData);*

fseek(cfPtr, offset, SEEK_SET);

- **Why (account - 1)?** File Byte Positions:

- Byte 0: Record #1 (Account 1)
- Byte 100: Record #2 (Account 2)
- Byte 200: Record #3 (Account 3)

- **Calculation:** $(3 - 1) \times 100 = \text{Byte } 200 \checkmark$

```

13 int main(void) {
14     FILE *cfPtr = NULL; // accounts.dat file pointer
15
16     // fopen opens the file; exits if file cannot be opened
17     if ((cfPtr = fopen("accounts.dat", "rb+")) == NULL) {
18         puts("File could not be opened.");
19     }
20     else {
21         // create clientData with default information
22         struct clientData client = {0, "", "", 0.0};
23
24         // require user to specify account number
25         printf("%s", "Enter account number (1 to 100, 0 to end input): ");
26         scanf("%d", &client.account);
27
28         // user enters information, which is copied into file
29         while (client.account != 0) {
30             // user enters last name, first name and balance
31             printf("%s", "Enter lastname, firstname, balance: ");
32
33             // set record lastName, firstName and balance value
34             fscanf(stdin, "%14s%9s%lf", client.lastName,
35                 client.firstName, &client.balance);
36
37             // seek position in file to user-specified record
38             fseek(cfPtr, (client.account - 1) *
39                 sizeof(struct clientData), SEEK_SET);
40
41             // write user-specified information in file
42             fwrite(&client, sizeof(struct clientData), 1, cfPtr);
43
44             // enable user to input another account number
45             printf("%s", "\nEnter account number: ");
46             scanf("%d", &client.account);
47         }
48
49         fclose(cfPtr); // fclose closes the file
50     }
51 }

```

```

Enter account number (1 to 100, 0 to end input): 37
Enter lastname, firstname, balance: Barker Doug 0.00

Enter account number: 29
Enter lastname, firstname, balance: Brown Nancy -24.54

```

The fread() Function

- **Function Prototype**

`size_t fread(void *ptr, size_t size, size_t count, FILE *stream);`

- **Reading a Single Record**

- `struct clientData client;`

- `fread(&client, sizeof(struct clientData), 1, cfPtr);`

- **What Happens**

- Reads `sizeof(struct clientData)` bytes
- From current file position pointer location
- Stores data in client variable
- Returns number of items successfully read

```

13 int main(void){
14     FILE *cfPtr = NULL; // accounts.dat file pointer
15
16     // fopen opens the file; exits if file cannot be opened
17     if ((cfPtr = fopen("accounts.dat", "rb")) == NULL) {
18         puts("File could not be opened.");
19     }
20     else {
21         printf("%-6s%-16s%-11s%10s\n", "Acct", "Last Name",
22             "First Name", "Balance");
23
24         // read all records from file (until eof)
25         while (!feof(cfPtr)) {
26             // read a record
27             struct clientData client = {0, "", "", 0.0};
28             size_t result =
29                 fread(&client, sizeof(struct clientData), 1, cfPtr);
30
31             // display record
32             if (result != 0 && client.account != 0) {
33                 printf("%-6d%-16s%-11s%10.2f\n", client.account,
34                     client.lastName, client.firstName, client.balance);
35             }
36         }
37
38         fclose(cfPtr); // fclose closes the file
39     }
40 }

```

Acct	Last Name	First Name	Balance
29	Brown	Nancy	-24.54
33	Dunn	Stacey	314.33
37	Barker	Doug	0.00
88	Smith	Dave	258.34
96	Stone	Sam	34.98

Case Study - Transaction Processing System

- **Program Capabilities**
- **Create formatted account list (Option 1)**
- **Update existing accounts (Option 2)**
- **Add new accounts (Option 3)**
- **Delete accounts (Option 4)**
- **Exit program (Option 5)**


```
5  #include <stdio.h>
6
7  // clientData structure definition
8  struct clientData {
9      int account;
10     char lastName[15];
11     char firstName[10];
12     double balance;
13 };

14
15 // prototypes
16 int enterChoice(void);
17 void textFile(FILE *readPtr);
18 void updateRecord(FILE *fPtr);
19 void newRecord(FILE *fPtr);
20 void deleteRecord(FILE *fPtr);
21
```

```

22 int main(void) {
23     FILE *cfPtr = NULL; // accounts.dat file pointer
24
25     // fopen opens the file; exits if file cannot be opened
26     if ((cfPtr = fopen("accounts.dat", "rb+")) == NULL) {
27         puts("File could not be opened.");
28     }
29     else {
30         int choice = 0; // user
31
32         // enable user to specify action
33         while ((choice = enterChoice()) != 5) {
34             switch (choice) {
35                 case 1: // create text file from record file
36                     textFile(cfPtr);
37                     break;
38                 case 2: // update record
39                     updateRecord(cfPtr);
40                     break;
41                 case 3: // create record
42                     newRecord(cfPtr);
43                     break;
44                 case 4: // delete existing record
45                     deleteRecord(cfPtr);
46                     break;
47                 default: // display message for invalid choice
48                     puts("Incorrect choice");
49                     break;
50             }
51         }
52
53         fclose(cfPtr); // fclose closes the file
54     }
55 }

```

```

192 // enable user to input menu choice
193 int enterChoice(void) {
194     // display available options
195     printf("%s", "\nEnter your choice\n"
196         "1 - store a formatted text file of accounts called\n"
197         "    \"accounts.txt\" for printing\n"
198         "2 - update an account\n"
199         "3 - add a new account\n"
200         "4 - delete an account\n"
201         "5 - end program\n? ");
202
203     int menuChoice = 0; // variable to store user
204     scanf("%d", &menuChoice); // receive choice from user
205     return menuChoice;
206 }

```

```

57 // create formatted text file for printing
58 void textFile(FILE *readPtr) {
59     FILE *writePtr = NULL; // accounts.txt file pointer
60
61     // fopen opens the file; exits if file cannot be opened
62     if ((writePtr = fopen("accounts.txt", "w")) == NULL) {
63         puts("File could not be opened.");
64     }
65
66     else {
67         rewind(readPtr); // sets pointer to beginning of file
68         fprintf(writePtr, "%-6s%-16s%-11s%10s\n",
69             "Acct", "Last Name", "First Name", "Balance");
70
71         // copy all records from random-access file into text file
72         while (!feof(readPtr)) {
73             // create clientData with default information
74             struct clientData client = {0, "", "", 0.0};
75             size_t result =
76                 fread(&client, sizeof(struct clientData), 1, readPtr);
77
78             // write single record to text file
79             if (result != 0 && client.account != 0) {
80                 fprintf(writePtr, "%-6d%-16s%-11s%10.2f\n", client.account,
81                     client.lastName, client.firstName, client.balance);
82             }
83         }
84
85         fclose(writePtr); // fclose closes the file
86     }
}

```



```

88 // update balance in record
89 void updateRecord(FILE *fPtr) {
90     // obtain number of account to update
91     printf("%s", "Enter account to update (1 - 100): ");
92     int account = 0; // account number
93     scanf("%d", &account);
94
95     // move file pointer to correct record in file
96     fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);
97
98     // read record from file
99     struct clientData client = {0, "", "", 0.0};
100    fread(&client, sizeof(struct clientData), 1, fPtr);
101
102    // display error if account does not exist
103    if (client.account == 0) {
104        printf("Account #%d has no information.\n", account);
105    }
106    else { // update record
107        printf("%-6d%-16s%-11s%10.2f\n\n", client.account, client.lastName,
108            client.firstName, client.balance);
109
110        // request transaction amount from user
111        printf("%s", "Enter charge (+) or payment (-): ");
112        double transaction = 0.0; // transaction amount
113        scanf("%lf", &transaction);
114        client.balance += transaction; // update record balance
115
116        printf("%-6d%-16s%-11s%10.2f\n", client.account, client.lastName,
117            client.firstName, client.balance);
118
119        // move file pointer to correct record in file
120        fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);
121
122        // write updated record over old record in file
123        fwrite(&client, sizeof(struct clientData), 1, fPtr);
124    }
125 }

```

```

127 // create and insert record
128 void newRecord(FILE *fPtr) {
129     // obtain number of account to create
130     printf("%s", "Enter new account number (1 - 100): ");
131     int account = 0; // account number
132     scanf("%d", &account);
133
134     // move file pointer to correct record in file
135     fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);
136
137     // read record from file
138     struct clientData client = {0, "", "", 0.0};
139     fread(&client, sizeof(struct clientData), 1, fPtr);
140
141     // display error if account already exists
142     if (client.account != 0) {
143         printf("Account #%d already contains information.\n",
144             client.account);
145     }
146     else { // create record
147         // user enters last name, first name and balance
148         printf("%s", "Enter lastname, firstname, balance\n? ");
149         scanf("%14s%9s%lf", &client.lastName, &client.firstName,
150             &client.balance);
151
152         client.account = account;
153
154         // move file pointer to correct record in file
155         fseek(fPtr, (client.account - 1) * sizeof(struct clientData),
156             SEEK_SET);
157
158         // insert record in file
159         fwrite(&client, sizeof(struct clientData), 1, fPtr);
160     }
161 }

```

```

163 // delete an existing record
164 void deleteRecord(FILE *fPtr) {
165     // obtain number of account to delete
166     printf("%s", "Enter account number to delete (1 - 100): ");
167     int account = 0; // account number
168     scanf("%d", &account);
169
170     // move file pointer to correct record in file
171     fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);
172
173     // read record from file
174     struct clientData client = {0, "", "", 0.0};
175     fread(&client, sizeof(struct clientData), 1, fPtr);
176
177     // display error if record does not exist
178     if (client.account == 0) {
179         printf("Account %d does not exist.\n", account);
180     }
181     else { // delete record
182         // move file pointer to correct record in file
183         fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);
184
185         struct clientData blankClient = {0, "", "", 0}; // blank client
186
187         // replace existing record with blank record
188         fwrite(&blankClient, sizeof(struct clientData), 1, fPtr);
189     }
190 }
191

```

- **Option 1: Create a Formatted List of Accounts**

Acct	Last Name	First Name	Balance
29	Brown	Nancy	-24.54
33	Dunn	Stacey	314.33
37	Barker	Doug	0.00
88	Smith	Dave	258.34
96	Stone	Sam	34.98

- **Option 2: Update an Account**

```
Enter account to update (1 - 100): 37
37      Barker          Doug          0.00

Enter charge (+) or payment (-): +87.99
37      Barker          Doug          87.99
```

- **Option 3: Create a New Account**

```
Enter new account number (1 - 100): 22  
Enter lastname, firstname, balance  
? Johnston Sarah 247.45
```


Compiling Multiple-Source-File Programs

- **Multi-File Program Structure**

myprogram/

- |— main.c # Main program entry point
- |— utils.c # Utility functions implementation
- |— database.c # Database operations
- |— main.h # Main header file
- |— utils.h # Utility function prototypes
- |— database.h # Database function prototypes

- **Key Rules**

- **Function definitions** must be entirely in **one file**
- **Cannot split** a single function across multiple files
- **Separate compilation** then linking

Global Variables Across Files

- **The extern Keyword**

- *// In file1.c*

- *int globalCounter = 0; // Definition*

- *// In file2.c*

- *extern int globalCounter; // Declaration (not definition)*

- *// Can now use globalCounter in file2*

- **How It Works**

- File1.c: `int flag = 1;` —┐

- | Linker connects

- File2.c: `extern int flag;` └┐

- `extern`: "This variable exists somewhere else"

- **Linker's job**: Match declarations with definitions

- **Error if**: Linker can't find definition

extern Example

- **Project Structure**

// config.c - Variable definition

```
int debugMode = 1;
```

```
float taxRate = 0.075;
```

// main.c - Using external variables

```
#include <stdio.h>
```

```
extern int debugMode;  // Declare, not define
```

```
extern float taxRate;  // Declare, not define
```

```
int main() {
```

```
    if (debugMode) {        printf("Debug mode ON\n");    }
```

```
    printf("Tax rate: %.2f%%\n", taxRate * 100);
```

```
    return 0;}
```

Compilation Command

bash

```
gcc main.c config.c -o program
```


Function Prototypes Across Files

- **Extending Function Scope**

// In mathutils.c

```
double calculateAverage(double arr[], int size) {  
    // Implementation  
}
```

// In mathutils.h

```
double calculateAverage(double arr[], int size); // Prototype
```

// In main.c

```
#include "mathutils.h" // Get prototype  
// Can now call calculateAverage()
```

How #include Works

```
#include <stdio.h>
```

```
// Copies printf, scanf prototypes into your file
```

```
// Linker finds the actual implementations in library
```

The static Keyword (Restricting Scope)

- **static Global Variables**

// In file1.c

static int internalCounter = 0; // Only visible in file1.c

// In file2.c

extern int internalCounter; // ERROR: Can't access!

// Linker will not find this variable

- **static Functions**

// In utils.c

static void helperFunction() { // Internal use only

// Cannot be called from other files

}

void publicFunction() { // Can be called from anywhere

helperFunction(); // OK - same file

}